

Symulacja pracy Szpitalnego Oddziału Ratunkowego

Dokumentacja projektu
Programowanie obiektowe

16 czerwca 2026

Spis treści

1	Wprowadzenie	2
2	Cel i zakres projektu	2
2.1	Cel główny	2
2.2	Zakres funkcjonalny	2
3	Opis problemu i motywacja	2
4	Architektura rozwiązania	3
4.1	Przegląd modułów	3
4.2	Przebieg symulacji	3
4.3	Hierarchia klas	3
5	Diagramy	4
6	Zastosowane mechanizmy obiektowe	5
7	Konfiguracja symulacji	5
8	Wyniki symulacji i panel analityczny	6
9	Wykorzystane narzędzia i technologie	6
10	Budowanie i uruchamianie	7
10.1	Kompilacja i uruchomienie symulacji	7
10.2	Testy	7
10.3	Panel analityczny	7
10.4	Dokumentacja	7

1 Wprowadzenie

Niniejszy dokument opisuje projekt przygotowany na zaliczenie z przedmiotu programowanie obiektowe. Głównym założeniem projektu jest zaprojektowanie i zaimplementowanie aplikacji w języku C++, która w sposób uproszczony odwzorowuje pracę Szpitalnego Oddziału Ratunkowego (SOR). W implementacji zastosowano podstawowe mechanizmy programowania obiektowego: dziedziczenie, polimorfizm, enkapsulację oraz kompozycję obiektów.

Projekt nie stanowi pełnego systemu informatycznego szpitala, lecz model dydaktyczny. Symulacja generuje losowych pacjentów, przydziela im priorytet w triażu, kolejkuje ich według pilności i obsługuje w gabinetach lekarskich. Wyniki zapisywane są do pliku CSV, a następnie można je przeanalizować w prostym panelu webowym napisanym w Pythonie.

2 Cel i zakres projektu

2.1 Cel główny

Celem projektu jest stworzenie symulacji dyskretniej, która pozwala obserwować, jak zmienia się obciążenie oddziału ratunkowego w czasie. Model umożliwia odpowiedź na kilka prostych pytań badawczych:

- ile pacjentów dociera do SOR-u w zadanym okresie,
- jak rośnie kolejka oczekujących,
- ile gabinetów jest jednocześnie zajętych,
- ile osób zostaje ostatecznie wyleczonych.

2.2 Zakres funkcjonalny

Program realizuje następujące zadania:

- wczytanie parametrów symulacji z pliku konfiguracyjnego `config.ini`,
- losowe generowanie pacjentów o różnych typach klinicznych,
- triaż wykonywany przez pielęgniarkę (przypisanie priorytetu),
- kolejkovanie pacjentów w poczekalni według priorytetu,
- leczenie w gabinetach konsultacyjnych prowadzonych przez lekarzy,
- symulację pogorszenia stanu pacjentów oczekujących (zależnie od typu),
- logowanie stanu systemu do pliku CSV po każdym kroku czasowym,
- wizualizację wyników w panelu Streamlit.

3 Opis problemu i motywacja

Tematem projektu jest Szpitalny Oddział Ratunkowy, czyli miejsce, w którym pacjenci przyjmowani są w trybie pilnym. W rzeczywistości personel musi szybko ocenić stan pacjenta (triaz) i zdecydować, kto wymaga natychmiastowej pomocy. W modelu projektu proces ten uproszczono do trzech poziomów priorytetu: *zielony*, *żółty* oraz *czerwony*.

4 Architektura rozwiązania

4.1 Przegląd modułów

Aplikacja składa się z kilku logicznych części:

Symulacja C++ Główna logika programu. Klasa `EmergencyDepartment` koordynuje przebieg symulacji.

Model danych Hierarchia klas reprezentujących osoby, pacjentów, pracowników medycznych oraz pomieszczenia.

Konfiguracja Plik INI z czasem symulacji, krokiem czasowym, liczbą gabinetów i prawdopodobieństwem pojawienia się pacjenta.

Logger CSV Zapis metryk: minuta, zajęte gabinety, liczba oczekujących, łączna liczba wyleczonych.

Panel analityczny (Python) Skrypt `dashboard.py` wczytuje CSV i rysuje wykresy w Streamlit.

Testy jednostkowe Testy Google Test sprawdzają wybrane klasy.

Dokumentacja API Dokumentacja kodu generowana przez Doxygen.

4.2 Przebieg symulacji

Symulacja działa w pętli czasowej. W każdym kroku (domyślnie co kilka minut wirtualnych):

1. system próbuje wygenerować nowego pacjenta (z zadanyim prawdopodobieństwem),
2. pielęgniarka wykonuje triaż i pacjent trafia do poczekalni,
3. aktualizowany jest stan pacjentów oczekujących,
4. w gabinetach kontynuowane jest leczenie,
5. wolne gabinety z przypisanym lekarzem przyjmują kolejnych pacjentów z poczekalni,
6. logger zapisuje bieżące statystyki.

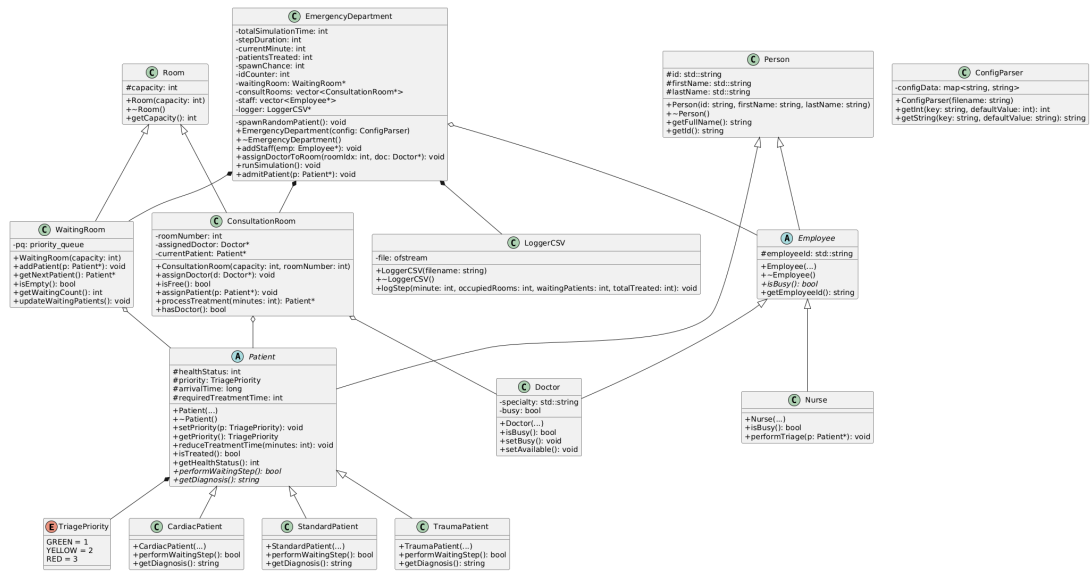
Kolejka w poczekalni jest implementowana jako kopiec priorytetowy. Pacjenci o wyższym priorytecie są obsługiwani wcześniej.

4.3 Hierarchia klas

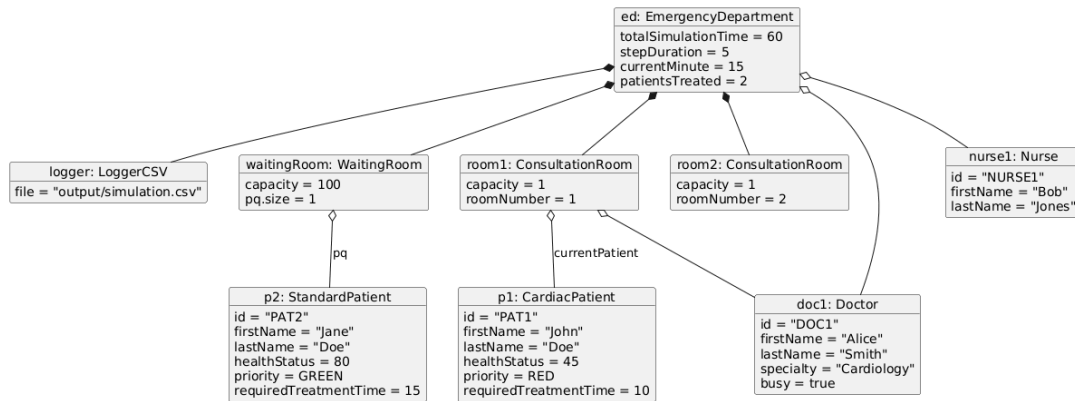
Poniżej przedstawiono najważniejsze relacje między klasami (szczegóły w diagramach w dalszej części dokumentu):

- **Person**: klasa bazowa dla ludzi w systemie.
- **Patient**: pacjent; klasa abstrakcyjna z metodami wirtualnymi `performWaitingStep()` oraz `getDiagnosis()`.
- **StandardPatient**, **CardiacPatient**, **TraumaPatient**: konkretne typy pacjentów.
- **Employee**: pracownik; klasa abstrakcyjna z metodą `isBusy()`.
- **Doctor**, **Nurse**: role medyczne.
- **Room**: pomieszczenie; bazowa dla **WaitingRoom** i **ConsultationRoom**.
- **EmergencyDepartment**: agreguje poczekalnię, gabinety, personel i logger.
- **ConfigParser**, **LoggerCSV**: klasy pomocnicze.

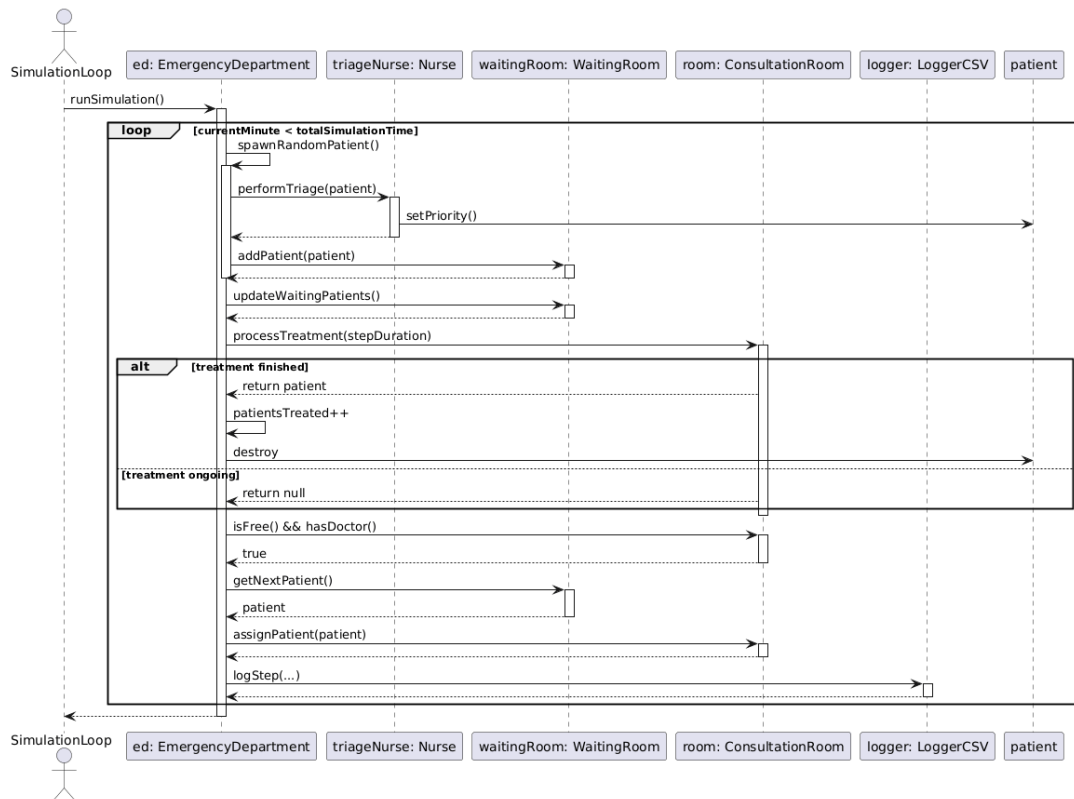
5 Diagramy



Rysunek 1: Diagram klas systemu symulacji SOR



Rysunek 2: Diagram obiektów (przykładowa konfiguracja w czasie triażu)



Rysunek 3: Diagram sekwencji: przyjęcie i obsługa pacjenta

6 Zastosowane mechanizmy obiektowe

W projekcie wykorzystano kilka koncepcji omawianych na zajęciach:

- **Dziedziczenie:** na przykład `Patient` dziedziczy po `Person`, a `CardiacPatient` po `Patient`.
- **Polimorfizm :** różne typy pacjentów inaczej zachowują się podczas oczekiwania (`performWaitingStep`).
- **Abstrakcja:** klasy bazowe definiują interfejs, a szczegóły implementują klasy pochodne.
- **Kompozycja i agregacja:** `EmergencyDepartment` zarządza obiektami poczekalni, gabinetów i pracowników.
- **Enkapsulacja:** pola klas są ukryte w sekcji `private/protected`, dostęp zapewniają metody.
- **RTTI:** w triażu pielęgniarka wyszukiwana jest przez `dynamic_cast<Nurse*>`.

7 Konfiguracja symulacji

Parametry wczytywane są z pliku `config/config.ini`. Przykładowa zawartość:

```
total_time=2139
step_duration=5
spawn_chance=67
num_rooms=2
```

Tabela 1: Opis parametrów konfiguracyjnych

Parametr	Typ	Znaczenie
<code>total_time</code>	int	Czas trwania symulacji w minutach wirtualnych
<code>step_duration</code>	int	Długość jednego kroku symulacji
<code>spawn_chance</code>	int	Szansa (od 0 do 99) pojawienia się pacjenta w kroku
<code>num_rooms</code>	int	Liczba gabinetów konsultacyjnych

8 Wyniki symulacji i panel analityczny

Po uruchomieniu program zapisuje dane do `output/simulation.csv`. Każdy wiersz zawiera m.in. bieżącą minutę, liczbę zajętych gabinetów, pacjentów oczekujących oraz skumulowaną liczbę wyleczonych osób.

Panel `dashboard.py` (Streamlit + pandas) umożliwia szybki podgląd wykresów bez ręcznej analizy pliku. Pozwala to ocenić, czy na przykład zwiększenie liczby gabinetów skraca kolejkę po zmianie konfiguracji i ponownym uruchomieniu symulacji.

9 Wykorzystane narzędzia i technologie

Tabela 2: Stos technologiczny projektu

Narzędzie	Zastosowanie
C++	Implementacja silnika symulacji
GCC / g++	Kompilator
GNU Make	Automatyzacja budowania
Google Test	Testy jednostkowe
Python 3	Panel wizualizacji wyników
Streamlit, pandas	Interfejs webowy i analiza niezmiennie użytecznych danych CSV
Doxygen	Automatyczna dokumentacja kodu źródłowego
L ^A T _E X	Niniejsza dokumentacja projektowa
Nix	Reprodukowalne środowisko budowania i paczki

10 Budowanie i uruchamianie

10.1 Kompilacja i uruchomienie symulacji

W katalogu projektu, po wejściu do powłoki Nix (`nix develop`), symulację uruchamia się poleceniami:

```
make  
make run
```

Plik wykonywalny powstaje w `bin/symulacja`. Przed uruchomieniem warto sprawdzić parametry w `config/config.ini`.

10.2 Testy

Testy uruchamiane są poleceniem:

```
make test
```

10.3 Panel analityczny

Po wygenerowaniu pliku CSV:

```
make dashboard
```

10.4 Dokumentacja

Cała dokumentacja (Doxygen + PDF) budowana jest jedną paczką Nix:

```
nix build .#docs
```

Wynik instalacji zawiera:

- `result/doxygen/`: dokumentacja HTML z Doxygen,
- `result/docs.pdf`: niniejszy dokument w formacie PDF.